

LAB 2

CHUẨN BỊ DỮ LIỆU IoT CHO AI VÀ DEMO DEPLOY MODEL CƠ BẢN

Bài mẫu dùng dataset công khai UCI Occupancy Detection, xử lý bằng Jupyter Notebook và deploy model bằng FastAPI

Hạng mục	Nội dung
Môn học	Triển khai, phát triển ứng dụng AI và IoT
Vị trí lab	Sau Lab 1/Lab 1.1; trước Lab 3 Anomaly Detection, Lab 4 Forecasting, Lab 5 Inference Service
Case mẫu	Smart Classroom Occupancy & Air Quality
Dataset mẫu chính	UCI Occupancy Detection; có fallback sample cùng schema để chạy khi lớp không có Internet
Công cụ	Python, Jupyter Notebook, pandas, scikit-learn, joblib, FastAPI, uvicorn
Đầu ra bắt buộc	Dataset sạch, feature dataset, baseline model .joblib, metrics.json, decision_log.csv, API /predict chạy được
Thời lượng	03 tiết trên lớp + bài về nhà theo nhóm

I. Định vị Lab 2 trong lộ trình môn học

Lab 2 nối tiếp Lab 1 và Lab 1.1 nhưng không phụ thuộc tuyệt đối vào việc nhóm đã có dữ liệu riêng. Mục tiêu của Lab 2 là giúp sinh viên nhìn thấy một pipeline AIoT tối thiểu có thể chạy thật: dữ liệu IoT được chuẩn hóa, tạo thành dataset AI, train baseline, lưu model và deploy thành API local.

Lab/Buổi	Vai trò	Sản phẩm kế thừa/sản phẩm tạo ra
Lab 1	Thiết kế kiến trúc AIoT	Use-case, data fields, AI module, feedback loop
Lab 1.1	Mô phỏng luồng dữ liệu AIoT	Telemetry CSV/JSON, anomaly demo, prediction demo, decision log mô phỏng
Lab 2	Chuẩn bị dữ liệu IoT cho AI và deploy baseline đầu tiên	Dataset sạch, feature dataset, model .joblib, FastAPI /predict, decision_log.csv
Lab 3	Anomaly Detection	Module phát hiện bất thường có metric rõ hơn
Lab 4	Forecasting	Model dự báo xu hướng và rủi ro
Lab 5	Inference Service	API inference hoàn chỉnh hơn: validation, logging, versioning, monitoring

II. Nguồn dữ liệu công khai có thể dùng cho Lab 2

Bài mẫu mặc định dùng UCI Occupancy Detection vì gần với bối cảnh Smart Classroom/Smart Building, có dữ liệu cảm biến môi trường và nhãn occupancy rõ ràng. Các dataset khác dùng cho bài phát triển nâng cao hoặc bài về nhà.

Dataset	Nguồn/link	Loại dữ liệu & trường chính	Use-case, mapping schema, lưu ý
UCI Occupancy Detection	https://archive.ics.uci.edu/ml/datasets/Occupancy%2BDetection Mirror tải trực tiếp: https://github.com/LuisM78/Occupancy-detection-data	Multivariate time-series; date, Temperature, Humidity, Light, CO2, HumidityRatio, Occupancy.	Case: Smart Classroom/Smart Building. Dùng cho classification occupancy, decision tiết kiệm năng lượng, cảnh báo CO2. Mapping: timestamp=date; telemetry fields giữ nguyên; target=Occupancy. Lưu ý: dữ liệu có sẵn train/test; khi dùng thực tế cần kiểm tra timestamp và class imbalance.
UCI Appliances Energy Prediction	https://archive.ics.uci.edu/ml/datasets/Appliances%2Benergy%2Bprediction	Time-series regression; appliances energy use, temperature/humidity trong nhà, weather, time features.	Case: Smart Home/Smart Building Energy. Dùng forecasting/prediction năng lượng. Mapping: timestamp=date; target=Appliances; sensors=temperature/humidity/weather. Lưu ý: nhiều feature, cần tránh dùng nhầm biến tương lai.

Triển khai, phát triển ứng dụng AI và IoT | LAB 2

UCI Air Quality	https://archive.ics.uci.edu/dataset/387/air%2Bquality	Hourly air quality; gas sensor responses, CO/NOx/NO2, temperature, humidity.	Case: Environmental Monitoring/Air Quality Station. Dùng regression, anomaly, decision cảnh báo chất lượng không khí. Mapping: timestamp=Date+Time; sensors=gas/temperature/humidity. Lưu ý: có giá trị missing dạng mã đặc biệt, cần clean kỹ.
Intel Berkeley Research Lab Sensor Data	https://db.csail.mit.edu/labdata/labdata.html	Sensor telemetry từ 54 node; timestamp, moteid, temperature, humidity, light, voltage.	Case: Environmental Sensor Telemetry/WSN. Dùng anomaly, sensor stuck, missing, node health. Mapping: device_id=moteid; timestamp=time; sensors=temp/humidity/light/voltage. Lưu ý: dữ liệu nhiều node, cần nhóm theo device_id.
Numenta Anomaly Benchmark (NAB)	https://github.com/numenta/NAB	Nhiều time-series đơn biến có nhãn anomaly; phù hợp streaming anomaly detection.	Case: IoT anomaly benchmark. Dùng Lab 3 để đánh giá detector. Mapping: timestamp + value → telemetry; label anomaly → event_label. Lưu ý: không phải schema phòng học; cần ánh xạ lại use-case.
Skoltech Anomaly Benchmark (SKAB)	https://github.com/waico/SKAB	Industrial multivariate time-series từ testbed; có anomaly/change-point.	Case: Smart Factory/IIoT anomaly. Dùng anomaly detection công nghiệp. Mapping: timestamp + sensor channels; label anomaly. Lưu ý: phù hợp nhóm khá/giỏi vì nhiều sensor và bối cảnh công nghiệp.
NASA C-MAPSS Turbofan Engine Degradation	https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/	Predictive maintenance; multivariate time-series theo engine/cycle, operational settings, sensor channels.	Case: Predictive Maintenance/IIoT. Dùng RUL prediction, degradation trend, maintenance decision. Mapping: device_id=engine_id; timestamp/cycle=cycle; sensors=sensor channels; target=RUL. Lưu ý: khó hơn Lab 2; phù hợp mở rộng hoặc bài tập lớn.

III. Bối cảnh thực tế của bài mẫu

Một phòng học thông minh có cảm biến nhiệt độ, độ ẩm, ánh sáng và CO2. Hệ thống IoT cũ có thể hiển thị số liệu và bật/tắt thiết bị theo ngưỡng. Tuy nhiên, dữ liệu thực tế thường thiếu, nhiễu, sai timestamp, duplicate, outlier hoặc cảm biến bị kẹt. Nếu dữ liệu sai, model AI có thể dự đoán sai trạng thái phòng, sinh cảnh báo sai hoặc đề xuất điều khiển sai.

Trong Lab 2, sinh viên không chỉ làm pandas. Sinh viên phải tạo được một AI baseline có thể deploy local: model nhận telemetry mới qua API `/predict`, trả về xác suất occupancy, anomaly_score và decision/command_hint.

IV. Mục tiêu sau Lab 2

- Kiểm tra được schema telemetry cho dữ liệu IoT dùng cho AI.
- Làm sạch dữ liệu IoT: timestamp, duplicate, missing, outlier, sensor value bất hợp lý.
- Tạo feature dataset từ telemetry.
- Chia train/test theo thời gian để tránh rò rỉ dữ liệu.
- Train được baseline model đầu tiên và lưu thành file `.joblib`.
- Tạo anomaly_score đơn giản bằng Z-score để hỗ trợ decision layer.
- Deploy model local bằng FastAPI và test endpoint `/predict`.
- Sinh `decision\_log.csv` thể hiện output AI được chuyển thành cảnh báo/khuyến nghị/lệnh có kiểm soát.
- Giải thích được vì sao dữ liệu sai có thể làm hệ thống AIoT điều khiển sai.

V. Cấu trúc project code sinh viên sẽ nhận

Giảng viên phát cho sinh viên file ZIP đi kèm. Sau khi giải nén, project có cấu trúc:

```
lab2_aiot_public_dataset_deploy/
├── data/
│   └── DATA_SOURCES.md
```

```

├── occupancy_fallback_same_schema.csv
├── notebooks/
│   └── 01_data_prep_baseline_deploy_ready.ipynb
├── src/
│   ├── data_utils.py
│   ├── download_data.py
│   ├── run_training_pipeline.py
│   ├── app.py
│   ├── test_api.py
│   └── check_outputs.py
├── models/
│   └── occupancy_baseline.joblib
├── outputs/
│   ├── metrics.json
│   ├── decision_log.csv
│   └── figures/
├── docs/
│   ├── sample_payload_predict.json
│   └── teacher_checklist.md
├── README.md
└── requirements.txt

```

File/thư mục	Sinh viên cần hiểu gì?
notebooks/01_data_prep_baseline_deploy_ready.ipynb	Phần chạy chính trên lớp: tải/nạp data, check schema, clean, feature, train, metric, decision log, lưu model.
src/data_utils.py	Nơi chứa các hàm lõi. Sinh viên đọc để hiểu logic, nhóm khá/giỏi có thể sửa decision rule hoặc feature.
src/app.py	FastAPI app. Đây là phần deploy model cơ bản: load `.joblib`, khai báo `/health`, `/model-info`, `/predict`.
src/test_api.py	Client test API. Sinh viên chạy sau khi deploy để biết API có hoạt động không.
src/check_outputs.py	Checklist tự động: kiểm tra đủ file output, metric, decision log và model artifact.
models/occupancy_baseline.joblib	Model đã train, dùng để API load khi deploy.
outputs/decision_log.csv	Cầu nối AI → decision/feedback. Đây là artifact quan trọng của Lab 2.

VI. Hướng dẫn cài môi trường

Yêu cầu: Python 3.10 trở lên. Khuyến nghị dùng VS Code hoặc JupyterLab.

1. Windows PowerShell

```

cd lab2_aiot_public_dataset_deploy
python -m venv .env
.\env\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt

```

2. macOS/Linux

```

cd lab2_aiot_public_dataset_deploy
python3 -m venv .env
source .env/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt

```

Nếu không dùng notebook, có thể chạy nhanh pipeline bằng terminal:

```
python src/run_training_pipeline.py
python src/check_outputs.py
```

VII. Bài mẫu: Smart Classroom Occupancy & Air Quality

Bài mẫu dùng dataset UCI Occupancy Detection. Nếu máy có Internet, script tải dữ liệu public từ GitHub mirror của tác giả. Nếu không có Internet, project dùng fallback sample cùng schema để sinh viên vẫn chạy được end-to-end. Cách này giúp lớp học không bị kẹt vì mạng, nhưng vẫn yêu cầu nhóm dùng public dataset thật khi nộp bài về nhà.

Bước 1. Tải/nạp dataset

Chạy trong terminal để kiểm tra data source:

```
python src/download_data.py
```

Kết quả mong đợi: terminal in ra `Dataset source`, `Shape`, danh sách cột và vài dòng đầu.

Bước 2. Mở Jupyter Notebook

```
jupyter lab
```

Mở file:

```
notebooks/01_data_prep_baseline_deploy_ready.ipynb
```

Chọn menu `Run → Run All Cells`. Sinh viên đọc từng cell, không chỉ bấm chạy.

Bước 3. Theo dõi các cell quan trọng trong notebook

Nhóm cell	Sinh viên cần quan sát	Dấu hiệu đúng
Import & path	PROJECT_ROOT có đúng thư mục project không?	In ra đường dẫn project, không lỗi import `src.data_utils`.
Load dataset	Đang dùng public dataset hay fallback sample?	Có `dataset_source`, dataframe có các cột date, Temperature, Humidity, Light, CO2, HumidityRatio, Occupancy.
Check schema	Thiếu cột, duplicate, số dòng?	`missing_columns` rỗng.
Clean data	Outlier/missing/duplicate được xử lý thế nào?	Có `telemetry_clean.csv`; số dòng sau clean hợp lý.
Feature dataset	Feature nào đưa vào model?	Có `feature_dataset.csv`; có cột `hour`, `dayofweek`.
Train/test split	Có chia theo thời gian không?	Train nằm trước Test theo timestamp.
Train model	Model gì, output gì?	Có metric: accuracy, precision, recall, f1, confusion matrix.
Decision log	AI output biến thành quyết định gì?	Có `decision`, `command_hint`, `safety_note`.
Save artifacts	Model đã sẵn sàng deploy chưa?	Có `models/occupancy_baseline.joblib`.

Bước 4. Kiểm tra output sau notebook

Sau khi notebook chạy xong, sinh viên phải thấy các file sau:

```
data/telemetry_clean.csv
data/feature_dataset.csv
models/occupancy_baseline.joblib
outputs/metrics.json
outputs/decision_log.csv
outputs/figures/01_co2_time_series.png
outputs/figures/02_confusion_matrix.png
outputs/figures/03_occupancy_probability.png
```

Chạy kiểm tra tự động:

```
python src/check_outputs.py
```

Kết quả đúng phải có dòng:

```
PROJECT CHECK PASSED: Notebook outputs and model artifacts are complete.
```

Bước 5. Deploy model bằng FastAPI

Mở terminal tại thư mục project và chạy:

```
uvicorn src.app:app --reload --host 127.0.0.1 --port 8000
```

Mở trình duyệt:

```
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/health
http://127.0.0.1:8000/model-info
```

Lưu ý cho sinh viên: deploy ở đây nghĩa là model không còn nằm trong notebook nữa. Model đã được lưu ra `joblib`, API load model và phục vụ request mới.

Bước 6. Test API /predict

Mở terminal thứ hai, vẫn ở thư mục project:

```
python src/test_api.py
```

Kết quả đúng phải có dòng:

```
API TEST PASSED: FastAPI model deployment is working.
```

Payload mẫu gửi vào API:

```
{
  "room_id": "room_101",
  "device_id": "env_node_01",
  "timestamp": "2015-02-06 06:32:00",
  "Temperature": 21.93,
  "Humidity": 28.786,
  "Light": 469.53,
  "CO2": 834.85,
  "HumidityRatio": 0.001824
}
```

Response mẫu cần có các phần:

```
{
  "model_output": {
    "occupancy_probability": ...,
    "predicted_occupancy": ...,
    "anomaly_score": ...,
    "is_anomaly": ...
  },
  "decision": {
    "decision": "...",
    "command_hint": "...",
  }
}
```

```

    "safety_note": "...
  }
}
```

VIII. Logic xử lý dữ liệu và model trong bài mẫu

Khâu	Logic mẫu	Artifact tạo ra
Raw telemetry	Dữ liệu từ UCI Occupancy hoặc fallback cùng schema.	DataFrame raw
Schema check	Kiểm tra các cột bắt buộc và duplicate.	schema_report
Clean	Parse timestamp, xóa duplicate, xử lý outlier, nội suy missing.	data/telemetry_clean.csv
Feature	Tạo hour, dayofweek, rolling mean/delta để giải thích.	data/feature_dataset.csv
Train/test	Chia theo thời gian 75/25.	train_df, test_df
Baseline prediction	Logistic Regression dự đoán Occupancy.	occupancy_probability, predicted_occupancy
Anomaly score	Z-score theo train statistics.	anomaly_score, is_anomaly
Decision rule	Kết hợp prediction + anomaly + CO2/light.	decision_log.csv
Deploy	FastAPI load model .joblib và phục vụ /predict.	API response JSON

IX. Nhiệm vụ thực hành bắt buộc trên lớp

1. Cài môi trường và chạy được notebook bài mẫu.
2. Xác định được dataset source đang dùng: public dataset hay fallback sample.
3. Giải thích được ít nhất 03 lỗi dữ liệu IoT đã xử lý trong notebook.
4. Chụp màn hình metric và confusion matrix.
5. Mở được FastAPI docs tại `http://127.0.0.1:8000/docs`.
6. Chạy được `python src/test_api.py` và lưu response `/predict`.
7. Mở `decision_log.csv` và giải thích 03 dòng quyết định khác nhau.

X. Yêu cầu phát triển theo nhóm và bài về nhà

Mức	Yêu cầu	Gợi ý sản phẩm
Bắt buộc	Dùng chính bài mẫu UCI Occupancy Detection. Chạy lại notebook, đổi ít nhất 01 decision rule, giải thích vì sao.	Notebook đã chạy, metrics.json, decision_log.csv, ảnh API /docs, response /predict.
Khá	Dùng dataset nhóm đã có từ Lab 1.1 hoặc chọn 01 dataset public khác trong danh sách. Ảnh xạ dữ liệu về schema AIoT gồm timestamp, device_id/room_id, sensors, target/event_label.	data_mapping.md, feature_dataset.csv, model.joblib, API /predict có input/output phù hợp use-case.
Giỏi	So sánh 02 baseline model hoặc thêm endpoint <code>/batch-predict</code> ; bổ sung safety rule để chặn điều khiển tự động khi anomaly_score cao/confidence thấp.	Bảng so sánh metric, API mở rộng, decision_log có confidence/safety_reason, phân tích edge/backend trade-off.

XI. Checklist hoàn thành

- ☐ Notebook chạy hết không lỗi.
- ☐ Có `data/telemetry_clean.csv`.
- ☐ Có `data/feature_dataset.csv`.
- ☐ Có `models/occupancy_baseline.joblib`.
- ☐ Có `outputs/metrics.json` gồm accuracy, precision, recall, f1.

- ☐ Có `outputs/decision\_log.csv` gồm occupancy_probability, anomaly_score, is_anomaly, decision, command_hint, safety_note.
- ☐ Có ít nhất 03 hình/biểu đồ trong `outputs/figures/`.
- ☐ API `/health` trả `model\_loaded: true`.
- ☐ API `/model-info` trả feature_cols và metrics.
- ☐ API `/predict` nhận JSON telemetry và trả model_output + decision.
- ☐ `python src/test\_api.py` chạy thành công.
- ☐ Sinh viên giải thích được model được deploy như thế nào: notebook train model → lưu .joblib → FastAPI load model → trả prediction cho request mới.

XII. Sản phẩm nộp

STT	Sản phẩm	Yêu cầu
1	Notebook đã chạy	Nộp file `.ipynb` hoặc PDF export, có output cell.
2	Source code	Nộp project code hoặc link GitHub; không nộp mỗi báo cáo.
3	Dataset/feature	`telemetry_clean.csv`, `feature_dataset.csv`.
4	Model artifact	`occupancy_baseline.joblib` hoặc model của use-case nhóm.
5	Decision log	`decision_log.csv` có output AI và decision/command_hint.
6	Mình chứng deploy	Ảnh chụp `/docs`, `/health`, response `/predict`, kết quả `python src/test_api.py`.
7	Báo cáo kỹ thuật ngắn	2-4 trang: dataset, schema, clean, model, metric, decision rule, rủi ro dữ liệu sai.

XIII. Rubric chấm điểm 10 điểm

Tiêu chí	Mô tả	Điểm
Dữ liệu và schema	Chọn đúng nguồn dữ liệu, kiểm tra schema, giải thích mapping về AIoT telemetry.	1.5
Xử lý dữ liệu IoT	Làm sạch timestamp, duplicate, missing, outlier; có giải thích ảnh hưởng đến AI.	1.5
Feature dataset và split	Tạo feature hợp lý, chia train/test theo thời gian, tránh leakage.	1.0
AI baseline	Train được model baseline, có metric phù hợp, lưu model artifact.	1.5
Anomaly score & decision log	Có anomaly_score/is_anomaly và decision/command_hint/safety_note rõ ràng.	1.5
Deploy model cơ bản	FastAPI chạy được, có `/health`, `/model-info`, `/predict`, test API thành công.	1.5
Giải thích kỹ thuật	Trình bày được dữ liệu → model → API → decision → feedback; nhận diện rủi ro dữ liệu sai.	1.0
Mở rộng	Tùy chỉnh use-case/dataset/rule/model hoặc bổ sung safety/endpoint nâng cao.	0.5

XIV. Câu hỏi tư duy/phản biện

8. Vì sao dữ liệu IoT time-series không nên chia train/test ngẫu nhiên?
9. Nếu cảm biến CO2 bị kẹt ở một giá trị cố định, model occupancy có thể sai như thế nào?
10. Tại sao API không nên tự động gửi lệnh điều khiển khi `is\_anomaly = true`?
11. Trong response `/predict`, trường nào thuộc output AI, trường nào thuộc decision layer?
12. Nếu model trả xác suất occupancy 0.55, có nên gửi lệnh bật điều hòa không? Cần thêm điều kiện gì?
13. Khi chuyển từ fallback sample sang public dataset hoặc dữ liệu nhóm, những phần code nào cần kiểm tra lại?
14. Lab 5 sẽ nâng cấp phần deploy này ở những điểm nào?

XV. Ghi chú cho giảng viên

Sinh viên chạy notebook trước, sau đó mới giải thích code. Không bắt đầu bằng FastAPI ngay vì sinh viên chưa quen deploy model. Sau khi có `.joblib`, model đã được đóng gói ra khỏi notebook; API chỉ load model và phục vụ request mới.

Trong môi trường kiểm thử, code đã được chạy end-to-end bằng fallback sample cùng schema do không có Internet. Khi lớp học có Internet, `src/download\_data.py` sẽ ưu tiên tải dataset UCI Occupancy Detection từ GitHub mirror của tác giả. Metric có thể khác nhau giữa fallback sample và public dataset; không yêu cầu sinh viên đạt cùng con số tuyệt đối, chỉ yêu cầu pipeline chạy đúng và giải thích được.

Phụ lục A. Kết quả kiểm thử

Các lệnh đã kiểm thử thành công:

```
LAB2_OFFLINE=1 python src/run_training_pipeline.py
python src/check_outputs.py
uvicorn src.app:app --host 127.0.0.1 --port 8000
python src/test_api.py
```

Các dấu hiệu thành công đã kiểm tra:

- `PROJECT CHECK PASSED: Notebook outputs and model artifacts are complete.`
- `API TEST PASSED: FastAPI model deployment is working.`
- `/health` trả `model\_loaded: true`.
- `/predict` trả `model\_output` và `decision`.